SymPy Bug Report

1 Bug 53: singularities Returns EmptySet for zeta(x) at Its Pole

1.1 Minimal Reproducer

```
from sympy import S, limit, symbols, zeta
from sympy.calculus.singularities import singularities

x = symbols("x")
result = singularities(zeta(x), x, S.Complexes)

print("singularities(zeta(x), x, S.Complexes) =", result)
print("1 in result =", result.contains(S.One))
print("zeta(1) =", zeta(1))
print("limit(zeta(x), x, 1) =", limit(zeta(x), x, 1))
```

1.2 Observed Behavior

```
singularities(zeta(x), x, S.Complexes) = EmptySet
1 in result = False
zeta(1) = zoo
limit(zeta(x), x, 1) = zoo
```

SymPy reports that $\zeta(x)$ has no singularities over the complex domain, while other SymPy evaluations in the same reproducer confirm that the value and limit at $x = 1$ are complex infinity. Thus the returned set is not merely an alternate representation: it omits a point that should be present.

1.3 Expected Behavior

The singularity set for $\zeta(x)$ over $\mathbb{C}$ should contain 1. For this input, the mathematically correct result is the singleton set $\{1\}$, since the Riemann zeta function has its isolated pole at $x = 1$.

1.4 Mathematical Explanation

The Riemann zeta function is meromorphic on the complex plane with a single simple pole at $s = 1$. Equivalently, near $s = 1$,

$$\zeta(s) = \frac{1}{s-1} + O(1),$$

so

$$\lim_{s \to 1} \zeta(s) = \infty$$

in the extended complex sense. Therefore, when the expression is $\zeta(x)$, the point $x = 1$ is a genuine isolated singularity. Returning $\emptyset$ asserts that no such point exists, which contradicts both the standard analytic behavior of ζ and SymPy's own evaluation $\zeta(1) = \texttt{zoo}$.

1.5 Source-Level Diagnosis

The diagnosis localizes the omission to `sympy/calculus/singularities.py`, specifically the implementation of `singularities` around lines 94–108. The traced call path enters the public `singularities` function, sympifies the input, rewrites selected trigonometric and hyperbolic functions, then scans only a limited set of syntactic forms: negative powers, zeros of arguments to `log`, `asech`, and `acsch`, and ± 1 arguments to `atanh` and `acoth`.

Special functions such as `zeta` are not inspected by this pass. For $\zeta(x)$, the expression has no relevant negative `Pow` atom and no atom belonging to the listed elementary functions, so the accumulator for singularities remains `S.EmptySet` and is returned unchanged. Unsupported special functions therefore appear to contribute no singularities instead of causing an incomplete-method signal.

The diagnosis also notes that the zeta implementation itself records the relevant pole behavior in `sympy/functions/special/zeta_functions.py`: $\zeta(1)$ evaluates to `S.ComplexInfinity`, and the function documentation states the simple pole at $s = 1$. The failure is therefore not that SymPy lacks all knowledge of the pole, but that `singularities` does not consult that knowledge. A plausible fix direction identified by the diagnosis is to add a special-function singularity hook, or otherwise have `singularities` consult metadata for known isolated poles; for $\zeta(s)$, this would solve $s - 1 = 0$ and include that solution in the result. If no such hook exists for an encountered special function, the diagnosis suggests preferring `NotImplementedError` over silently returning an incomplete `EmptySet`.

1.6 Additional Failing Instances

The independent verification checks the representative case and five shifted arguments of the same pole. For $\zeta(x+a)$, the zeta pole occurs when $x + a = 1$, so the expected singular point is $x = 1 - a$. The buggy behavior is the same across the family: `singularities` returns a set that does not contain the expected pole.

Input / Expression	**SymPy behavior**	**Expected behavior**
$\zeta(x)$	Does not contain 1; observed representative result is $\emptyset$	Contains 1
$\zeta(x+1)$	Does not contain 0	Contains 0
$\zeta(x+2)$	Does not contain -1	Contains -1
$\zeta(x-1)$	Does not contain 2	Contains 2
$\zeta(x+3)$	Does not contain -2	Contains -2
$\zeta(x-2)$	Does not contain 3	Contains 3

These cases are all instances of the same analytic fact: composing ζ with an affine argument shifts the simple pole from $s = 1$ to the solution of the affine equation $x + a = 1$.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material found was SymPy documentation for `singularities`, which documents limitations and the distinction between `EmptySet` and `NotImplementedError`, a broad plotting issue about better singularity handling, and an unrelated zeta-derivative recursion issue. None of the available evidence identified a public report for singularities$(\zeta(x), x, \mathbb{C})$, the omission of $\zeta(1) = \texttt{zoo}$, or this special-function-pole omission. The deduplication verdict is therefore a new failure mode with no public precedent, and the case remains individually actionable as a focused issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import S, symbols, zeta
from sympy.calculus.singularities import singularities


@pytest.mark.parametrize("offset", [0, 1, 2, -1, 3, -2])
def test_singularities_zeta_includes_shifted_pole(offset):
    x = symbols("x")
    result = singularities(zeta(x + offset), x, S.Complexes)
    assert result.contains(S.One - offset) is S.true
```